Revature

Bank of CLI

Project 1, Team 3

Revature

A reliable front-end for a bank experience.

We built a single-page application for Bank of CLI that is ready to integrate with a future backend

October 2026

Technology Stack

1. Angular – Single Page Application Framework (SPA)
2. TypeScript – Typed Models and Predictable Data Structures
3. HTML – Structure & semantic elements for the website.
4. CSS – Interactive/Responsive Styling and Presentation
5. JSON – Mock API responses for the Service Layer
6. Git/Github – Version Control and Team Collaboration

Architecture

A description of our service-based architecture

1

Component layer

The component layer is responsible for the user-facing parts of our application and how users interact with them

2

Service layer

The service layer handles the application's business logic and communication between the components and backend systems.

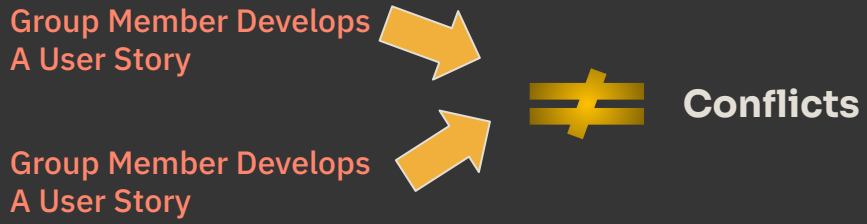
3

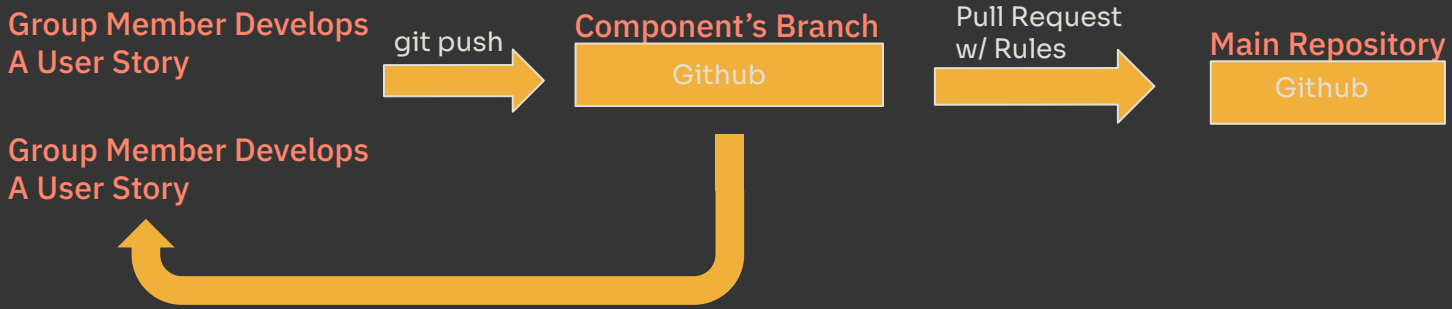
Contract layer

The contract layer defines the structure and format of the data exchanged between different parts of the application.

Git/Github Utilization

How did collaboration amongst us happen?





Revature

Frontend Experience

An overview of our responsive, interactive web dashboard

1

Secure access

The user is prompted to log into their account. Pages such as the dashboard and transactions page can't be accessed when not logged in.

2

Dashboard

The user has access to their balance, recent transactions and a navigation bar hosting 3 transactions.

3

Transaction center

The transaction center contains the Deposit Withdraw and Transfer functions of the Bank. It will prompt the user with a popup UI.

4

Professional UX

Features smooth animations, pop ups, easy navigation with back buttons, user-readable error messages, UI responsiveness, and a consistent style.

Revature

Authentication

How users register, login, and logout

Register

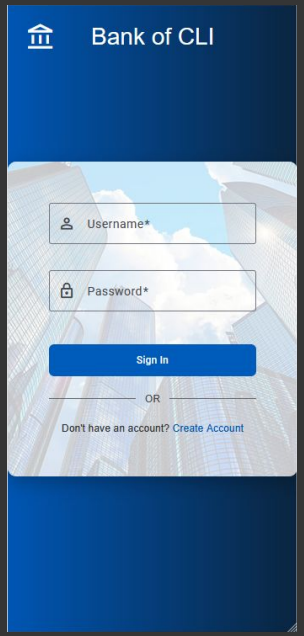
Creates a new account

→ Loads starter accounts from a mock Json file.
→ Auto Generates each accounts id
→ Sends the user to the login once the account is created

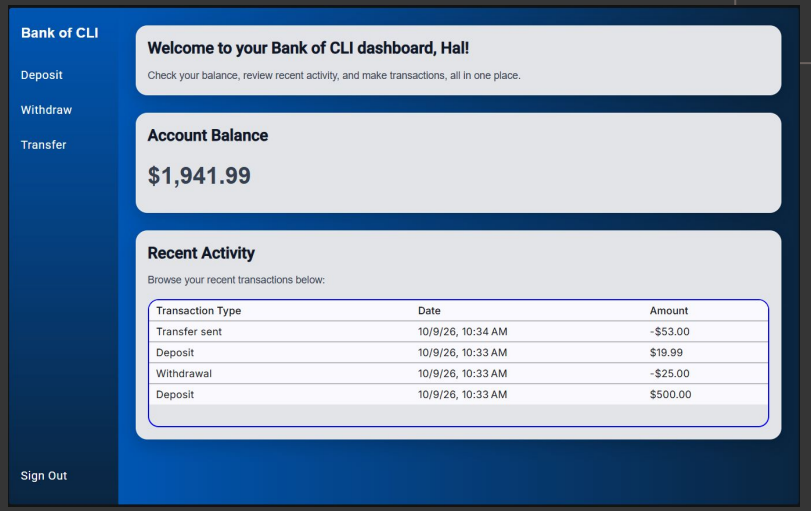
Login & Logout

Signs the user into account

→ saves the logged-in a signal the whole app can read
→ Sends the user to their dashboard on success
→Sign out clears the logged-in user and returns them to log in



Menu Animation



→ Menu only appears while a user is logged in.
→ Waits for the slide-out to finish before removing the menu
→ On sign out, the menu slides off before the user returns to login

Ethan Duarte

Registration

How users can register an account

Component

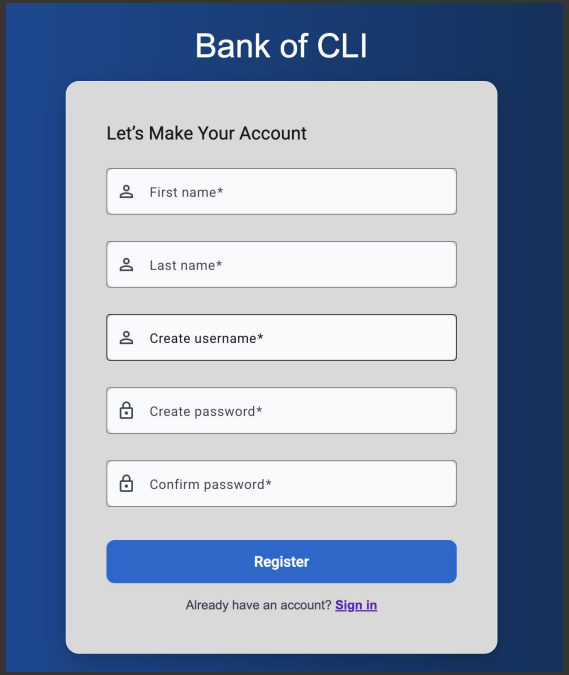
→ User Interface for the Login and Registration pages was developed using Angular Material components such as MatFormField, MatInput, MatButton, MatCard, and MatIcon.

→ User information is entered through Reactive Forms, allowing values such as username and password to be collected and validated before being passed to the application's service layer.

U.I. Implementation

→ Login and Registration page styling, including colors, spacing, form fields, buttons, and card layout, is implemented within each component's CSS.

→ Like other components, the Login and Registration pages are connected through Angular routing, allowing users to navigate between the two pages within the Single Page Application.



Transaction Center

	Deposit	Withdraw	Transfer
Overview	Allows the user to add money into their balance.	Allows the user to take out money from their balance.	Allows users to send money to another user's account.
Features	➔ Prevents invalid inputs. ➔ Provides a responsive UI for the user to interact with.	➔ Carries over features from deposit. ➔ Ensures users have enough balance for the transaction.	➔ Carries over features from both deposit and withdraw. ➔ Allows the user to input the recipient's account with validation checks.
Validation	➔ Ensures the user inputted a positive number greater than $0.00.	➔ Same number input validation as Deposit. ➔ Prevents user from inputting a number greater than the balance.	➔ Same input validation as both Deposit and Withdraw. ➔ Ensures the recipient's account input is valid and that the account exist.

Transaction Table

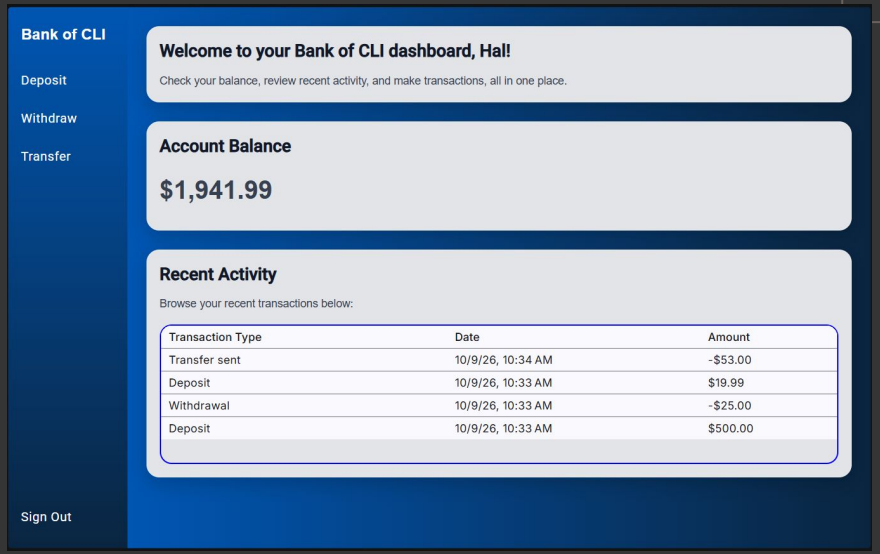
How users can view their transactions

Component

→ User Interface for the transaction was developed using the MatTableModule from the Angular Material Library.
→ User's transactions will appear row by row, with a max of 5 rows before the ability to scroll is allowed to see the additional rows.

U.I. Implementation

→ Transaction Table's style (size, font, padding, etc.) is implemented within the component's CSS.
→ Like other components, placement of the component is made inside the encompassing dashboard component.



Transaction Service

Requirement: The Service Layer “handles all data logic” and components should ask a Service for it

One Service owns the logic

Deposit, withdraw, transfer and transaction history all live in TransactionService

Rules first, then the change

All rules are checked before any balance changes. A rejected request changes nothing

One of two results

Returns the transaction or an error message from our contract. Confirmed by 17 tests

```
deposit(request: DepositRequest): StandardTransaction | ApiError {  Show usages  ksaylesjr
  const account = this.auth.currentUser();

  // rule: you can only deposit into the account you are logged in to
  if (!account || account.id !== request.accountId) {
    return this.error(401, 'You must be logged in to make a transaction.');
  }

  // rule: the amount must be a whole number of cents, above zero
  if (!Number.isInteger(request.amount) || request.amount <= 0) {
    return this.error(400, 'The transaction amount must be greater than zero.');
  }

  // rule: a single deposit cannot exceed the limit
  if (request.amount > MAX_TRANSACTION_AMOUNT) {
    return this.error(400, 'The deposit/transfer amount exceeds the maximum allowed limit of $10,000.');
  }

  // do the math here, then hand AuthService the new balance
  this.auth.updateBalance(account.username, account.balance + request.amount);
```

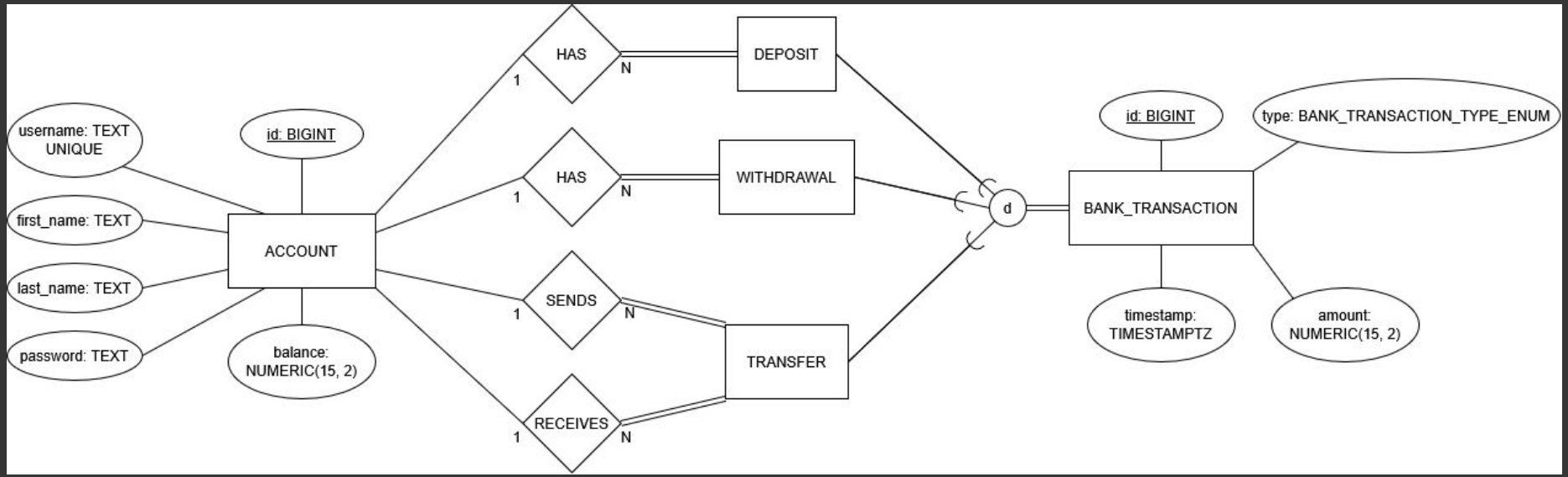
Rules enforced: logged in - within the $10,000 limit - sufficient funds - recipient exists

API Contract

The structure of the data exchanged between the frontend and the service

EER Diagram

- To inform our data structure and API design, we modeled a database in advance
- Database structure does not directly affect frontend, but we took it into account for designing the contract layer



API Contract

The structure of the data exchanged between the frontend and the service

Request: Transfer Money

```
{
    "senderAccountId": 51,
    "recipientAccountId": 52,
    "amount": 51
}
```

Successful Response: Get Transfer

```
{
    "id": 51,
    "senderAccountId": 51,
    "recipientAccountId": 52,
    "type": "TRANSFER",
    "amount": 51,
    "timestamp": "2024-06-01T00:00:00Z"
}
```

Unsuccessful Response: Transfer Recipient Does Not Exist

```
{
  "status": 404,
  "message": "The recipient account could not be found.",
  "timestamp": "2024-06-01T00:00:00Z"
}
```

JSON Structure

- We defined consistent JSON structures for:
  - Requests
  - Successful Responses
  - Unsuccessful responses
- The frontend reads simulated backend responses that follow these structures
- Our future backend will also follow this contract, so it will connect seamlessly with the frontend

Demo

Questions?